Exam Assignment 2026

Machine Learning in Particle Physics and Astronomy

Start date: May 18, 2026
Deadline: July 12, 2026

This task is about the reconstruction of particle trajectories in a particle physics experiment, e.g. in the ATLAS experiment at the Large Hadron Collider at CERN or in astroparticle experiments like the KM3NeT/Icecube experiments. This is one of the most important experimental algorithms in astroparticle and particle physics. So far, conventional algorithms have generally been used, but Deep Learning will take over trajectory reconstruction in the future. However, the question is: what is the best approach and algorithm to reconstruct these particles with neural networks?

When particles collide in a particle detector, the collision generates a multitude of secondary particles. These particles can travel through the detector, leaving behind signals called **"hits"** as they interact with the detector material. The hits are recorded by various detector components such as tracking detectors.

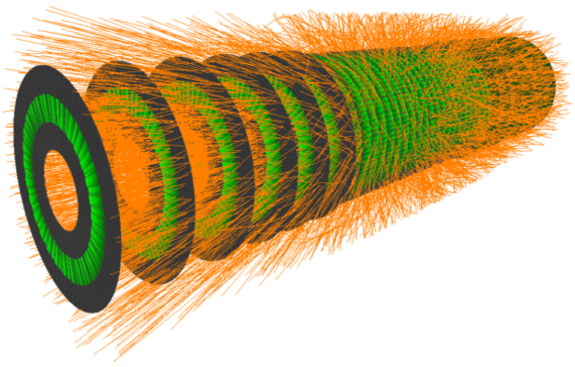
By reconstructing particle tracks from detector hits, scientists can determine the properties of the particles, such as their momentum, charge, and type, enabling further analysis and identification of specific particles, such as Higgs bosons or other exotic particles. The track reconstruction process plays a crucial role in unraveling the physics of particle collisions at experiments like the Large Hadron Collider.

The process of reconstructing particle **tracks** from these detector **hits** is essential for understanding the paths and properties of the particles produced in the collision. Here's a simplified explanation of the track reconstruction process:

The tracking detectors are designed to measure the trajectories of charged particles. They consist of layers of sensitive material, such as silicon detectors, which generate electrical signals when charged particles pass through them. These signals are the hits recorded by the detector.

The first step in track reconstruction is to associate hits that likely originate from the same particle. This process groups together hits that are spatially close to each other in the detector. Hits produced by different particles are expected to be well-separated, allowing for their identification as individual tracks. One problem is that the number of detector hits for each collision (or event) varies widely (from 100 to well over 10000) and the number of tracks also varies from a few to thousands. The image below shows a simulated collision event with many tracks (in orange) detected by a set of detectors (in green). The image is from the Kaggle machine learning tracking competition, see:

`https://www.kaggle.com/competitions/trackml-particle-identification`



In the traditional approach this clustering process is followed by the so-called Track Seed Generation: Once the hits are clustered, track seed generation begins. Track seeds are initial estimates of particle trajectories that are constructed using a subset of hits. Finally, the hits are used to determine the parameters that define the track candidates. This is usually done by a statistical fit. In general, the tracks are described by 4-5 parametric helix functions as the particles curve in the magnetic field.

Our simplified setup: In our simplified setup we have no magnetic field and all particles start from the origin (0,0). Then the particles are straight lines, i.e. they are described by only 1 parameter in 2 dimensions and 2 parameters in 3 dimensions.

Your first task is to write a simplified simulator for tracks and detector hits produced in a particle collision. Since random numbers are used to generate the collisions and detector responses, this is an example of a so-called Monte Carlo simulation.

Monte Carlo simulation is a computational technique that uses random sampling to model and analyze complex systems or processes. It involves generating many random events, simulating their behavior according to predefined physical rules or models, and analyzing the resulting data.

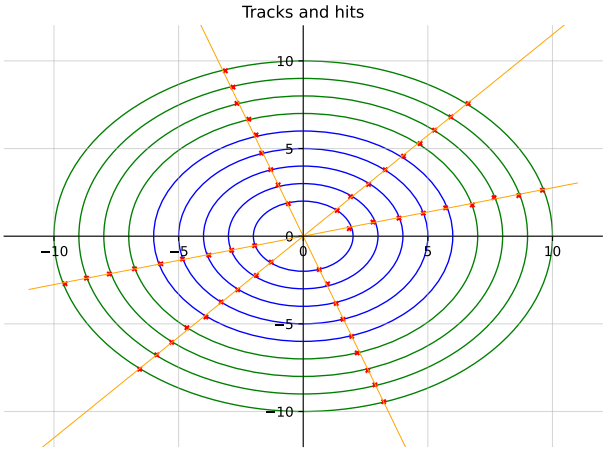
If you do not succeed in implementing the simulator completely, you may use an LLM/AI assistant (e.g. ChatGPT or Claude) to help you.

Your second task is to develop and test Deep Learning algorithms and architectures that can associate detector hits to the particle tracks that produced them. We will also provide a second dataset containing tracks in a magnetic field. In this case, the particle trajectories are curved, as shown in the figure on page 3. You can download the dataset from Brightspace. The dataset contains two files: one containing the detector hits and one containing the corresponding track information for each event. Again, your task is to associate the hits to the correct tracks.

Please note that the numbering of the tracks is arbitrary and has no physical meaning. Therefore, a model should not learn fixed track labels such as "track 1", "track 2", etc. Instead, the goal is to correctly group hits that belong to the same physical track.

In detail, your tasks are :

- **a) Create/generate events with random tracks in 2D** as lines from the origin, similar to the figure below. Start with the simple case that each event has exactly 3 different tracks. We use a simple detector geometry to describe the detector hits: Assume that the detectors are circular around the origin. Describe a hit as the point where the trace intersects with the detectors (described by circles). Use 5 circles with different radii (and equal distances between the radii) for the beginning (blue part).



- **b)** Due to detector effects, hits are only recorded with a 95 percent probability, i.e. the events will not all have the same number of hits as some might be missing. Furthermore, some detectors are not fully efficient and the precise position of the hits is not entirely known. To simulate the position measurements, a small random deviation is applied to the x and y positions of each hit. This deviation is modeled using Gaussian random noise with x and y values with a standard deviation of 0.1 percent, respectively.

  Add these two detector effects to your simulator and generate 10,000 events, each with 10 tracks and the simulated number of hits.

- **c) Create histograms** for the distribution of hits and tracks for your simulated events in x, y and phi (the angle relative to the x-axis).

- **d) Assign hits to tracks using a neural network.** Train a neural network that takes the detector hits of one event as input and assigns each hit to the track that produced it. This is a hit-to-track association problem.

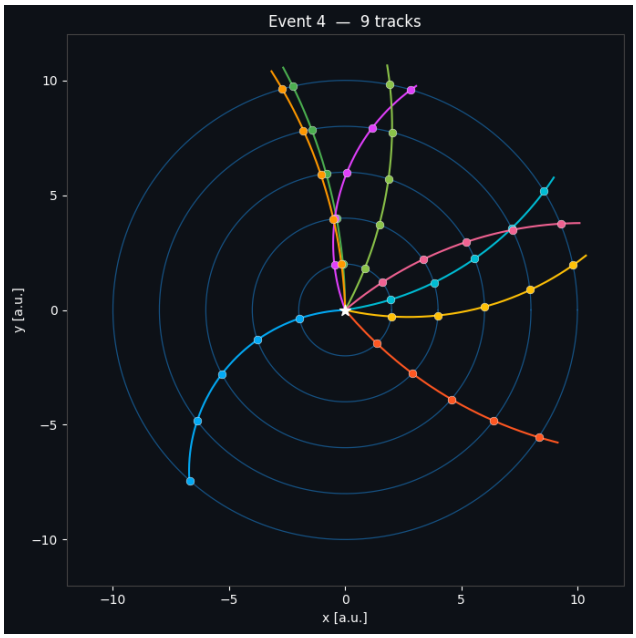
  Possible architectures include fully connected networks, recurrent networks, transformers, or graph neural networks. Since the number of hits can vary from event to event, you should explain how your network handles variable-size inputs.

  Evaluate the performance by comparing the predicted hit-to-track assignments with the true assignments from the simulation. Report the fraction of hits assigned to the correct track. Also show example events with true and predicted hit assignments. How does the solution scale if you increase to 50 tracks ?

- **e) Apply the method to the provided curved-track dataset.** Use the second dataset provided. In this dataset, each event contains a random number of tracks between 5 and 15. The tracks are curved due to a magnetic field. The curvature depends on the track momentum (the 2nd parameter of the problem, proportional to the curvature/radius of the track) and charge (positive or negative).

  Your task is again to associate hits to the tracks that produced them. Take into account that both the number of hits and the number of tracks vary from event to event. Also note that the numbering of the tracks is arbitrary, so the evaluation should be based on whether hits belonging to the same true track are grouped together correctly, not on the absolute numerical track labels.

  Evaluate your method again (determine the fraction of correctly assigned hits) and show example events with true and predicted hit-to-track associations.



- f) **BONUS POINT if you have a lot of computing resources: Try the same approach in 3d using spheres as detector elements. Now the tracks are described by 3 parameters, R, phi and theta.**

1 Handing in

The deadline for the assignment is printed at the top of the first page. Any assignment handed-in after this deadline will not be graded. Handing in your assignment is done through Brightspace. You need to hand in **four** elements. Not handing in all four before the deadline will result in us not grading your solution.

Element 1: A scientific paper describing your solution

Using your development logbook you use during the development of the algorithm, create a scientific paper about your solution. This write-up **should be maximum 4 pages long (appendix allowed and not counted towards the page limit)** and in the form of a letter like the ones published by Physical Review Letters (PRL) as a reference for this[1]. Use the following structure:

- Start with an abstract;
- Describe the problem;
- Describe the investigated methods and why you used these;
- Report and discuss your results;
- Conclude with a conclusion;

[1] For example: `https://journals.aps.org/prl/pdf/10.1103/PhysRevLett.122.014502`.

- Add an appendix with technicalities[2].

- Use references

We will discuss a bit more how to structure a scientific paper in the discussion sessions (17 May and a few weeks after, to be discussed via slack).

Element 2: Your code

Your code in .py or .ipynb format.

Element 3: The trained algorithms

The models you trained in saved format. What kind of file you hand in, depends on the used library:

- **keras**: Use the `.save` method of the keras model to save it to an .hdf5 file. See `https://keras.io/getting-started/faq/#how-can-i-save-a-keras-model`.
- **sklearn**: Use the built in **joblib** package in python to store the object containing your trained model. See `https://scikit-learn.org/stable/modules/model_persistence.html`.

Please also include a script to read and run your saved model(s).

Element 4: Predictions by your algorithms

For each of your algorithms, create a file with the data produced and provide the file on Brightspace. Put each generated event on its own line, prepended by the ID of the event the data belongs to. Separate the event data by a comma. A single line of your file might therefore look like this:

```
21561, hit1_x, hit1_y,  hit1_trackID_1, hit_2_x, hit2_y, hit2_trackID_2,  ...
```

You may have a second file with the track parameter for each event, i.e. event 21561 would have an entry like this:

```
21561, trackID_1, trackparameter1, trackID_2, trackparameter_2, ...
```

This information can be used to grade the performance of your algorithm.

2 Grading

The final grade will be constructed based on a grading of the paper and the code. The write-up, algorithm and performance contributes 40%, the code 20% of the final grade, whereas the oral discussion contributes 40%. The write-up will be checked on the following:

- Does it show the student understands the theoretical concepts of the applied methods?

- Does the choice for applied (machine learning) methods make sense?

- Is the student concise in argumentation?

- Given the argumentation and available hardware, did the student find an algorithm that suits the problem?

- Was any optimisation performed and, if so, was this optimisation done in a sound manner?

- Is the write-up written like a scientific paper?

The code will be graded based on the following criteria:

- Does the code do what the student described in the paper?

- Is the implementation of the solutions correct? (e.g. does it *run*?)

[2]PRL normally does not allow for appendices, but for the purpose of this assignment we allow you to add one anyway.

- Do the algorithms work and perform as described?
- Is the code readable (e.g. does it contain enough explanation in the form of comments for someone who has never read the code to understand it? Is it structured logically?)

The oral discussion will be graded based on the following criteria:

- Does the student understand the methods and results presented in the report?
- Can the student clearly explain the code, the proposed solutions, and the machine learning models used?